

CERTIFICACIÓN UNIVERSITARIA



UMC



FCEFYN



Manual para el
alumno

Jenkins II



WE WANT
SKILLS!

mundosE

Manual para el alumno: Jenkins II.....	2
Objetivo del encuentro:.....	2
1. Pipelines declarativos y tradicionales.....	3
1.1 De Freestyle a Pipeline.....	3
1.2 Scripted vs. Declarative.....	3
1.3 Anatomía de un Declarative Pipeline.....	4
1.4 Librerías compartidas.....	4
1.5 Buenas prácticas.....	4
2. Integración con Git y Maven.....	5
2.1 Credenciales y seguridad.....	5
2.2 Optimizando Maven.....	5
2.3 Reportes de calidad.....	5
2.4 Estrategia de ramas y PRs.....	6
2.5 Gestión de versiones.....	6
3. Despliegue de artefactos en AWS S3.....	6
3.1 Razonamiento arquitectónico.....	6
3.2 Políticas IAM de mínimo privilegio.....	6
3.3 Etapa DeployToS3 en Declarative.....	7
3.4 Costos y retención.....	7
3.5 Estrategias de <i>rollback</i>	7
4. Monitoreo con Prometheus.....	8
4.1 Prometheus Plugin.....	8
4.2 <i>Dashboards</i> en Grafana.....	8
4.3 Alertas recomendadas.....	9
4.4 Buenas prácticas de observabilidad.....	10
4.5 Integración con otras herramientas.....	10
Bibliografía.....	11

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este encontraremos un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el alumno: Jenkins II

Objetivo del encuentro:

1. Diferenciar los enfoques *Freestyle*, *Scripted* y **Declarative Pipeline**, señalando ventajas y desventajas de cada uno.
2. Configurar la integración continua entre Jenkins, Git y Maven empleando credenciales seguras y webhooks.
3. Diseñar un pipeline completo que compile, testee y publique artefactos en un bucket **AWS S3** siguiendo buenas prácticas de seguridad.
4. Implementar métricas de salud de Jenkins mediante **Prometheus Plugin** y razonar alertas básicas.

1. Pipelines declarativos y tradicionales

La evolución natural de un equipo que comienza con Jobs Freestyle culmina, casi inevitablemente, en **Pipeline as Code**. En Jenkins existen dos DSL principales: **Scripted Pipeline** (Groovy imperativo) y **Declarative Pipeline** (DSL estructurado). Entender sus diferencias es clave para seleccionar la técnica adecuada según el contexto.

1.1 De Freestyle a Pipeline

Los Jobs Freestyle resuelven rápidamente la automatización cuando se trata de un solo paso de compilación o despliegue. Sin embargo, crecen las necesidades y comienzan a aparecer limitaciones: falta de paralelismo nativo, dificultad para versionar la configuración y *click-ops drift* (desfase entre la UI y el código real). Para mitigar estos problemas, en 2016 se introdujo el **Pipeline Plugin** que permite describir el flujo completo dentro de un fichero **Jenkinsfile** alojado en el repositorio.

1.2 Scripted vs. Declarative

Característica	Scripted	Declarative
Sintaxis	Groovy puro (<i>try/catch</i> , bucles).	DSL con bloques fijos (<i>pipeline</i> , <i>stages</i> , <i>steps</i>).
Curva de aprendizaje	Alta (programación).	Media (estructura opinionada).
Validación	Solo en <i>runtime</i> .	Linter integrado en UI + CLI
Extensiones	Flexibilidad total.	Librerías compartidas + <i>when</i> , <i>options</i>

Declarative impone un esqueleto que fuerza buenas prácticas: todos los pasos deben vivir dentro de *stages*, cada *stage* posee *steps* y los *post conditions* (*always*, *success*, *failure*) evitan fugas de limpieza. Scripted entrega libertad total, útil cuando se requiere lógica compleja o manipulación dinámica de matrices.

1.3 Anatomía de un Declarative Pipeline

```
pipeline {
  agent { docker { image 'maven:3.9-eclipse-temurin-17' args '-v ~/.m2:/root/.m2' } }
  options { skipStagesAfterUnstable() disableConcurrentBuilds() }
  stages {
    stage('Checkout') {
      steps { git branch: 'main', url: 'git@github.com:org/app.git', credentialsId: 'git-creds' }
    }
    stage('Build') {
      steps { sh 'mvn -B clean package' }
    }
    stage('Test') {
      steps { sh 'mvn test' }
      post { always { junit '**/target/surefire-reports/*.xml' } }
    }
  }
  post {
    success { echo 'Build OK' }
    failure { echo 'Build FAIL' }
  }
}
```

Cada bloque es declarativo; la imagen Docker garantiza reproducibilidad, `skipStagesAfterUnstable()` evita continuar si una etapa marca *unstable* y `disableConcurrentBuilds()` previene colisiones.

1.4 Librerías compartidas

Para evitar repetición se definen funciones Groovy reusables alojadas en un repo aparte y declaradas con `@Library('org-shared-pipelines')`. Esto habilita *steps* como `mysteps.publishToS3()` y estandariza la cadena DevOps.

1.5 Buenas prácticas

1. Versionar `Jenkinsfile` en cada rama para reflejar cambios.
2. Usar `agent none` + `stage` con `agent` específico cuando distintas etapas requieren entornos diferentes.
3. Declarar *timeouts* (`options { timeout(time: 20, unit: 'MINUTES') }`).
4. Evitar secretos en texto plano; recurrir a `withCredentials`.
5. Documentar cada *stage* con comentarios breves.

Recursos para profundizar:

- A. **Jenkins** © (2025). Documentación oficial Declarative.
<https://www.jenkins.io/doc/book/pipeline/syntax/>
- B. **GitHub** © (2025). Ejemplo de librería compartida.
<https://github.com/jenkinsci/workflow-cps-global-lib-plugin#creating-libraries>

2. Integración con Git y Maven

La eficiencia de un *pipeline* depende tanto del código Jenkins como de la orquestación con el SCM y el sistema de *builds*. Git y Maven representan el tándem más frecuente en proyectos Java.

2.1 Credenciales y seguridad

Crear una credencial **SSH Username with private key** (o **Username/Password** si se usa HTTPS) en *Manage Jenkins* → *Credentials*. La ID se referencia en el *Jenkinsfile* (`credentialsId: 'git-creds'`). Para repositorios privados, configurar un *webhook* en GitHub apuntando a <https://<jenkins-url>/github-webhook/>. Esto dispara el *pipeline* sin necesidad de *polling*, reduciendo ruido.

2.2 Optimizando Maven

Descargar dependencias Maven es costoso la primera vez. Estrategias:

- **Cache compartido:** montar `~/.m2` como volumen host o PVC en Kubernetes.
- **Repository Manager:** Artifactory / Nexus que actúa como proxy y almacena jars localmente.
- **Parallel builds:** `MAVEN_OPTS="-T1C"` compila módulos en paralelo.

2.3 Reportes de calidad

El *plugin* **JUnit** consume XMLs de pruebas y grafica tendencias. Para cobertura de código, usar **JaCoCo Plugin** (`goal jacoco:report`). Los resultados aparecen en la barra lateral del *build*, permitiendo inspección rápida.

2.4 Estrategia de ramas y PRs

Multibranch Pipeline escanea un repo y crea automáticamente *jobs* por cada rama o Pull Request. Esto promueve un flujo *trunk-based* con revisiones aisladas, evitando que cambios inestables entren en **main**. El *status context* devuelve una marca verde/roja al PR en GitHub.

2.5 Gestión de versiones

El plugin **Maven Release** actualiza **pom.xml**, crea *tag* y despliega al repositorio. En *pipelines* declarativos conviene invocar **mvn -B release:prepare release:perform --settings settings.xml** dentro de un *stage* exclusivo para evitar colisiones.

3. Despliegue de artefactos en AWS S3

Publicar en S3 convierte a este servicio en un repositorio de artefactos altamente disponible y duradero.

3.1 Razonamiento arquitectónico

S3 ofrece *11 nines* de durabilidad y un modelo de objeto inmutable. Cada *build* puede subir el JAR con nombre **app-*<BUILD_NUMBER>*.jar**, permitiendo trazabilidad. Además, *S3 Event Notifications* posibilita encadenar *workflows*: una Lambda que actualiza un AMI o un pipeline de infraestructura.

3.2 Políticas IAM de mínimo privilegio

Crear usuario IAM **jenkins-artifact-publisher** con política limitada:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:PutObject", "s3:ListBucket"],
    "Resource": ["arn:aws:s3:::jenkins-lab-bucket", "arn:aws:s3:::jenkins-lab-bucket/*"]
  }]
}
```

Guardar **AWS_ACCESS_KEY_ID** y **AWS_SECRET_ACCESS_KEY** en Jenkins como credencial *Username/Password* (ID **AWSAccess**).

3.3 Etapa DeployToS3 en Declarative

```
stage('DeployToS3') {
  steps {
    withCredentials([usernamePassword(credentialsId: 'AWSAccess',
                                     usernameVariable: 'AWS_ACCESS_KEY_ID',
                                     passwordVariable: 'AWS_SECRET_ACCESS_KEY')]) {
      sh '''
        aws s3 mb s3://jenkins-lab-bucket || true
        aws s3 cp target/*.jar s3://jenkins-lab-bucket/ --metadata build=$BUILD_NUMBER
      '''
    }
  }
}
```

Tras el *upload*, una línea `aws s3 ls` verifica presencia del archivo.

3.4 Costos y retención

Almacenar *x builds* diarios *x* tamaño *y* MB puede inflar costos. Habilitar *Lifecycle Rules* que muevan objetos a **Glacier Instant Retrieval** tras 30 días reduce tarifa de almacenamiento 68 %. Revisar `aws s3api put-bucket-lifecycle-configuration`.

3.5 Estrategias de *rollback*

Con *versioning* activo, un simple `aws s3 cp s3://bucket/key?versionId=abc old.jar` restaura una versión anterior. Otra opción: un archivo JSON `manifest.json` que indique el artefacto “estable”; cambiarlo equivale a promocionar o revertir sin copiar grandes binarios.

Recursos para profundizar:

- A. **AWS** © (2025). What is Amazon S3? (Best practices for S3 cost optimization) <https://docs.aws.amazon.com/AmazonS3/latest/userguide/storage-class-analysis.html>
- B. **AWS** © (2025). Lifecycle configuration elements. (Ejemplo Lifecycle JSON) <https://docs.aws.amazon.com/AmazonS3/latest/userguide/intro-lifecycle-rules.html>

4. Monitoreo con Prometheus

La observabilidad cierra el ciclo DevOps: no basta con compilar y desplegar; se necesita medir la salud de la plataforma CI para anticipar cuellos de botella, dimensionar recursos y sostener acuerdos de nivel de servicio (SLA). Usar **Prometheus** —estándar de facto en métricas time-series— combinado con el **Prometheus Metrics Plugin** de Jenkins habilita una visión cuantitativa de todo el sistema.

4.1 Prometheus Plugin

Después de instalar el *plugin* y reiniciar Jenkins, el *endpoint* <http://<jenkins>/prometheus> expone unas 265 métricas en formato *OpenMetrics*. Entre las más relevantes:

Métrica	Descripción	Pregunta que responde
<code>jenkins_job_duration_seconds</code>	Histograma de duración por <i>job</i>	¿Qué <i>jobs</i> disparan alertas de tiempo excesivo?
<code>jenkins_queue_size_value</code>	Tamaño actual de la Build Queue	¿Necesitamos más ejecutores/agentes?
<code>jenkins_executor_utilization</code>	% de <i>executors</i> ocupados	¿Estamos sub-o sobre-provistos?
<code>jenkins_plugins_failed_value</code>	Plugins con estado FAILED	¿Hay componentes rotos tras un upgrade?

Todas las métricas incluyen etiquetas (*labels*) como `job="build-api"` o `result="SUCCESS"` que permiten cortes por proyecto o estado.

4.2 Dashboards en Grafana

Grafana se integra con Prometheus mediante *data source* nativo y ofrece plantillas como el *dashboard* **ID 9964 “Jenkins CI by Robinson”**: panel de *heatmap* de duración, barra de éxito/fallo semanal y gráfico de distribución de tiempos de cola. Para equipos pequeños alcanza con:

- Panel *Build Success Rate* (verde/rojo).

- Panel *Top 5 jobs más lentos* usando `topk(5, avg by (job) (rate(jenkins_job_duration_seconds_sum[1h]) / rate(jenkins_job_duration_seconds_count[1h])))`.
- Panel *Utilización de ejecutores* con `avg(jenkins_executor_utilization)`.

4.3 Alertas recomendadas

Regla	Expresión PromQL	Severidad	Justificación
Build queue alta	<code>jenkins_queue_size_value > 5</code> durante 5m	warning	Indica cuello de botella; quizá falten agentes.
Duración de job anómala	<code>increase(jenkins_job_duration_seconds_sum{job="build-api"}[15m]) / increase(jenkins_job_duration_seconds_count{job="build-api"}[15m]) > 1.5 * avg_over_time(jenkins_job_duration_seconds{job="build-api"}[6h])</code>	critical	Detecta <i>builds</i> que tardan 50 % más que la media.
Executor saturation	<code>jenkins_executor_utilization > 0.9</code> durante 10m	warning	Riesgo de colas largas y tiempos de entrega mayores.
Fallo de plugin	<code>jenkins_plugins_failed_value > 0</code>	critical	Un <i>plugin</i> roto puede bloquear builds o comprometer seguridad.

Las alertas se envían a Slack o email a través de **Alertmanager**, permitiendo ruteo por severidad y silenciado durante mantenimientos.

4.4 Buenas prácticas de observabilidad

1. **Etiqueta consistente:** agregar `team`, `service` a las métricas personalizadas.
2. **Retention policy:** guardar 30 días en Prometheus y archivar históricos en Thanos o Cortex para análisis longitudinal.
3. **Capacity planning:** rastrear la métrica `executor_utilization` y planificar el número de agentes necesarios para mantenerla < 70 % en hora pico.
4. **Pruebas A/B:** comparar duración del *pipeline* antes/después de optimizar Maven cache usando panel de percentiles.
5. **Budget alerts:** correlacionar `jenkins_job_duration_seconds` con costos de *cloud agents* (EC2 spot) para prevenir gastos inesperados.

4.5 Integración con otras herramientas

- **OpenTelemetry:** el *plugin* experimental envía *traces* para enlazar eventos de build con *spans* de microservicios.
- **Elasticsearch / Kibana:** exportar *logs* de consola vía Logstash para correlacionar fallos con *spikes* de duración.
- **Sentry:** anotar cada despliegue con `sentry-cli releases finalize` dentro de un *step* Post-Build para rastrear errores de aplicación respecto al build Jenkins.

Recursos para profundizar:

- A. **Github** © (2025). Documentación Prometheus Plugin.
<https://github.com/jenkinsci/prometheus-plugin>
- B. **Prometheus** © (2025). Alertmanager.
<https://prometheus.io/docs/alerting/latest/alertmanager/>

Bibliografía

Haines, P. (2023). *CI/CD con AWS y Jenkins: Automatización de despliegues en la nube de Amazon* (1ª ed.). Alfaomega Grupo Editor.

Lewis, J. (2024). *Pipeline as Code: Continuous Delivery with Jenkins, Kubernetes, and Terraform* (2ª ed.). Manning Publications.

Prometheus Authors. (2024). *Prometheus Monitoring Guide: Best Practices for Alerting and Visualization*. The Linux Foundation.

Smart, K. (2023). *Jenkins Essentials: Continuous Integration and Continuous Delivery With Jenkins* (3ª ed.). Packt Publishing.

The Jenkins Project. (2025). *Jenkins User Documentation* (LTS 2.466).
<https://www.jenkins.io/doc/>